ADVANCED DATA TECHNOLOGIES

Multiclass Support Vector Machines

One-vs-One versus One-vs-Rest on an imbalanced intrusion-detection problem

A Support Vector Machine is intrinsically a binary classifier; this report shows how it is wrapped to separate three classes, why the two wrapping strategies differ, and why on a deliberately imbalanced network-intrusion problem the headline accuracy of 0.90 hides a costly failure on the one class that matters most the rare exploit.

Quantity Result Reading

OvO decision_function columns 3 matches k(k-1)/2 = 3 OvR decision_function columns 3

matches k = 3 OvO cross-validated accuracy 0.8650 +/- 0.026 5-fold stratified OvR

cross-validated accuracy 0.8633 +/- 0.024 statistically tied with OvO Overall test accuracy

0.900 misleadingly healthy **Exploit-class (2) recall 0.615 38% of real exploits missed** Table

The data: a deliberately imbalanced intrusion set

The dataset is a synthetic three-class network-intrusion problem generated with scikit-learn. The class weights are skewed on purpose so that the most important class is also the rarest the realistic situation in intrusion detection, where genuine attacks are a small fraction of traffic.

```
X, y = make_classification(
    n_samples=600, n_features=8, n_informative=5, n_redundant=0,
    n_classes=3, n_clusters_per_class=1, weights=[0.47, 0.45, 0.08],
    class_sep=0.7, flip_y=0.05, random_state=7)
```

Class Meaning Share Count (of 600) 0 benign traffic 47% 276 1 reconnaissance 45% 270 **2 genuine exploit (must not miss) 8% 54** Table 2 Class distribution. Class 2 is the rare, high-cost target.

Why an SVM needs a multiclass wrapper

A standard SVM finds a single separating hyperplane between two classes — the widest-street boundary from Sub-task 1. It has no native concept of three classes. To classify k classes the problem is decomposed into several binary problems whose answers are then combined. Two standard decompositions exist.

One-vs-One (OvO)

Train one classifier for **every pair** of classes. Each sees only its two classes; all other rows are ignored during its training. At prediction time every classifier votes for one of its two classes and the class with the most votes wins.

- Number of classifiers: **k(k-1)/2** → at k=3 that is three: (0 vs 1), (0 vs 2), (1 vs 2).
- Each sub-problem is **balanced and small** only two classes' worth of data.

One-vs-Rest (OvR)

Train one classifier **per class**, each asking ‘this class versus everything else’. The class whose classifier is most confident wins.

• Number of classifiers: **k** → at k=3 that is three: (0 vs rest), (1 vs rest), (2 vs rest). • Each sub-problem is **lopsided** the ‘rest’ side lumps every other class together, so the minority class is pitted against a far

larger combined negative set.

Figure 1
The two decompositions at k=3. Both yield three classifiers here, but OvO trains balanced pairs while OvR trains each class against a pooled rest.

The crossover at k = 3

The assignment fixes k=3 because it is the single value where the two formulas give the **same** count, so you can confirm they agree before they pull apart. The cost difference — OvO grows quadratically, OvR linearly only appears at four classes or more.

k	OvO = k(k-1)/2	OvR = k	Relationship
3	3	3	coincide
4	6	4	diverge
5	10	5	diverge

3 decision_function and a silent scikit-learn detail

decision_function(X) returns the raw pre-threshold confidence scores — one column per internal binary decision. Its column count reveals how the model is wired:

```
SVC(kernel="rbf", decision_function_shape="ovo").decision_function(X).shape[1] # -> 3 SVC(kernel="rbf", decision_function_shape="ovr").decision_function(X).shape[1] # -> 3
```

The catch. scikit-learn's SVC **always trains One-vs-One internally**, whatever you pass. The decision_function_shape argument only reshapes the returned scores (collapsing the OvO votes into OvR-style per-class scores); it does not change which classifiers are trained. To obtain a genuinely OvR-trained model you must wrap explicitly:

```
from sklearn.multiclass import OneVsRestClassifier
OneVsRestClassifier(SVC(kernel="rbf"))
```

That explicit wrapper is exactly why the accuracy comparison in §5 uses OneVsRestClassifier and not merely decision_function_shape="ovr".

4 Why scaling lives inside a Pipeline

An RBF SVM is **distance-based**: its kernel measures how close points are, so a feature on a large numeric scale would dominate the distance and swamp the others. Hence StandardScaler. But the scaler must be fit **inside** the cross-validation loop, never on the whole table first:

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
```

```
model = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
```

Scaling the full dataset before splitting lets the scaler ‘see’ the validation rows — their mean and variance leak into training. A Pipeline refits the scaler on each fold’s training rows only. This is the leak-free discipline carried over from S3.

OvO versus OvR: cross-validated accuracy

Both wrappers are evaluated with identical 5-fold stratified cross-validation, each inside its own scaling Pipeline. The genuine OvR model uses the explicit OneVsRestClassifier from §3.

Model CV accuracy Std OvO SVC(kernel="rbf") 0.8650 +/- 0.0260 OvR OneVsRestClassifier(SVC(rbf))

0.8633 +/- 0.0239

Per-class evaluation: where accuracy lies

Accuracy is the fraction of all predictions that are correct; on imbalanced data it is dominated by the majority classes. Here classes 0 and 1 are 95% of the data, so a model can score 0.90 while quietly failing on the 8% that matters. The honest view is the per-class confusion matrix on a held-out, stratified 25% test fold.

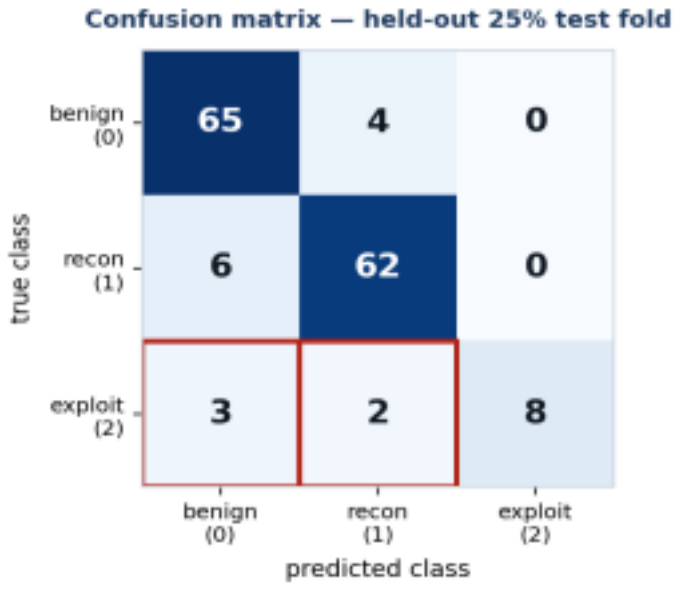

Figure 2 Confusion matrix (rows = true, columns = predicted). The red box marks the costly cell: 5 of 13 real exploits filed as benign or recon.

From the matrix we derive two metrics per class:

• **Precision** of everything the model called class C, how much really was C? (false-alarm view) • **Recall** of all the real class-C cases, how many did the model catch? (miss view)

Class Precision Recall F1 Support 0 benign 0.878 0.942 0.909 69 1 recon 0.912 0.912 0.912 68 **2**

exploit 1.000 0.615 0.762 13 Table 5 Per-class report. Class 2 has perfect precision but recall of only

0.615.

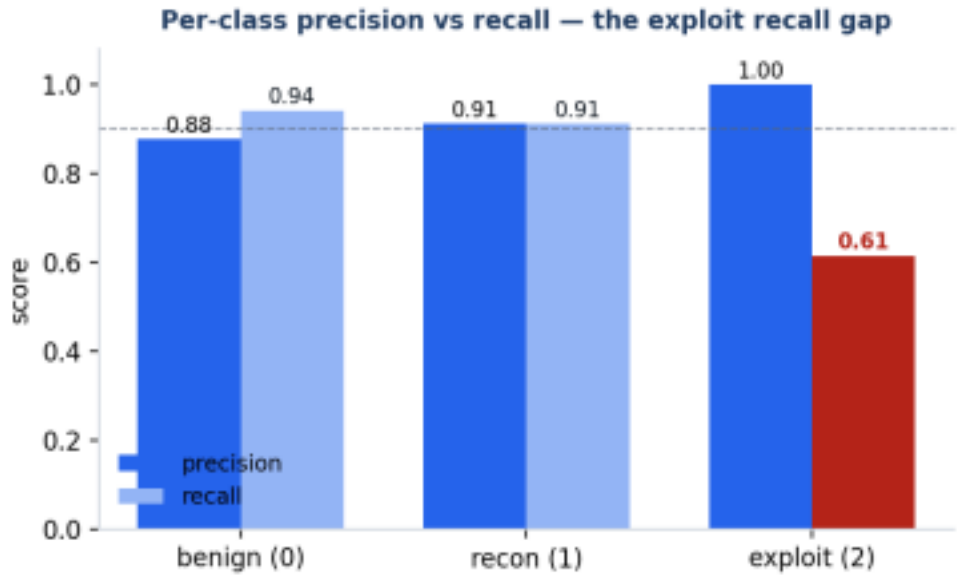

Figure 3 — Precision (dark) vs recall (light) per class. The exploit class never produces a false alarm but is missed nearly 4 times in 10 — the red recall bar. Dashed line marks overall accuracy (0.90).

Class 2 shows **perfect precision but poor recall**: when the model says ‘exploit’ it is always right, but it stays silent on nearly four in ten real exploits, filing them as benign or recon instead. Accuracy of 0.90 looks healthy precisely because the two easy majority classes carry it.

Cost-aware choice

In intrusion detection the two error types are not equally expensive. A **false positive** on class 2 (benign flagged as exploit) costs an analyst a few minutes of review — cheap. A **false negative** on class 2 (a real exploit filed as benign) means the attack goes undetected — expensive. The metric to optimise is therefore **class-2 recall**, and the headline accuracy is actively misleading because it is propped up by the two large, easy classes.

Wrapper decision. OvO (0.8650) and OvR (0.8633) are a statistical tie, so accuracy does not choose between them. I keep the default **OvO SVC**: identical accuracy at this k, and OvO trains on balanced binary pairs rather than pitting the tiny exploit class against the combined mass of everything else (the lopsided situation OvR creates), which is gentler on the minority recall that matters most.

Fixing class-2 recall (the right next step): set class_weight="balanced" to penalise misclassifying the rare class more heavily, or lower class 2’s decision threshold to trade some of its perfect precision for more catches. Both accept slightly more false alarms in exchange for missing fewer real exploits — the correct trade when a miss is the costly error.

Full reproducible code

Every number in this report reproduces exactly from the seeds shown (random_state=7 for the data, random_state=42 for the split and CV splitter).

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.svm import SVC
from sklearn.multiclass import OneVsRestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import train_test_split, StratifiedKFold,
cross_val_score from sklearn.metrics import confusion_matrix,
precision_recall_fscore_support

X, y = make_classification(
    n_samples=600, n_features=8, n_informative=5, n_redundant=0,
    n_classes=3, n_clusters_per_class=1, weights=[0.47, 0.45, 0.08],
    class_sep=0.7, flip_y=0.05, random_state=7)

# 1. decision_function column counts + formula check
ovo = make_pipeline(StandardScaler(), SVC(kernel="rbf",
decision_function_shape="ovo")).fit(X, y)
ovr = make_pipeline(StandardScaler(), SVC(kernel="rbf",
decision_function_shape="ovr")).fit(X, y)
print(ovo.decision_function(X).shape[1], ovr.decision_function(X).shape[1]) # 3 3

# 2. genuine OvO vs OvR cross-validated accuracy
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
ovo_clf = make_pipeline(StandardScaler(), SVC(kernel="rbf"))
ovr_clf = make_pipeline(StandardScaler(),
OneVsRestClassifier(SVC(kernel="rbf"))) print(cross_val_score(ovo_clf, X, y,
cv=cv).mean()) # 0.8650
print(cross_val_score(ovr_clf, X, y, cv=cv).mean()) # 0.8633

# 3. per-class confusion matrix on a held-out split
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.25, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), SVC(kernel="rbf")).fit(Xtr, ytr) ypred =
model.predict(Xte)
print(confusion_matrix(yte, ypred))
print(precision_recall_fscore_support(yte, ypred, labels=[0, 1, 2]))
```

Glossary

Term Meaning

OvO One-vs-One: one classifier per pair of classes, k(k-1)/2 total, majority vote. OvR One-vs-Rest: one classifier per class (class vs everything), k total. decision_function Raw pre-threshold confidence scores; column count reveals the wiring. Precision Of predicted-C, how many were really C (false-alarm view).

Recall Of real C, how many were caught (miss view).

Stratified split / CV Keeps each class's proportion in every fold; essential so rare class 2 appears in test. Leakage Letting validation data influence training (e.g. scaling before splitting); avoided by Pipeline.